Express.js

Complete Project Notes

Architecture · Code Walkthrough · Interview Prep

Based on: node-express-mongodb-demo project | Express 5 + MongoDB Native Driver

1. Project Overview

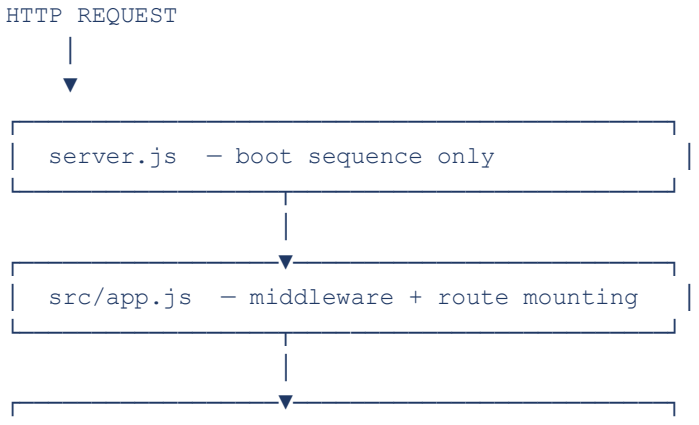
This project is a production-style REST API built with Express 5, MongoDB native driver, and ES Modules. It demonstrates the layered architecture pattern — the most common structure used in real Node.js backend projects.

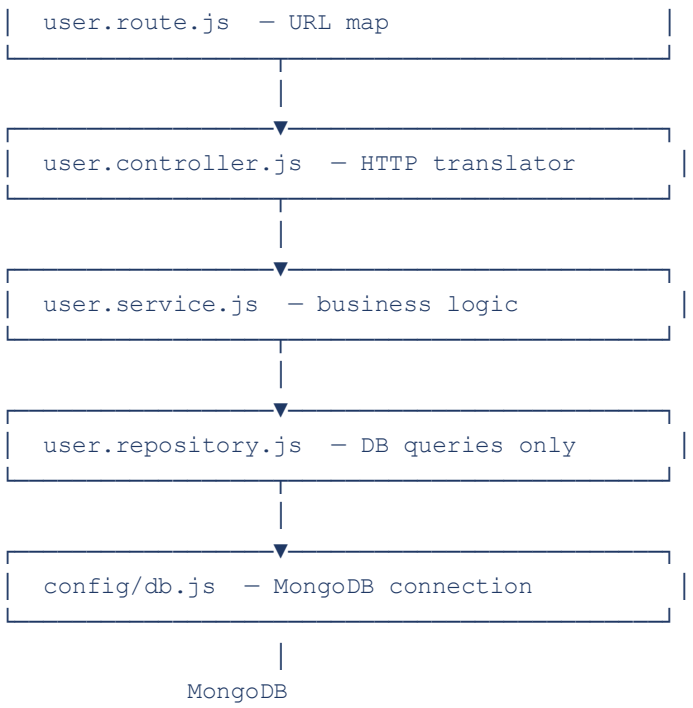
1.1 Folder Structure

```
node-express-mongodb-demo/
├── server.js                    ← Entry point (boot sequence only)
├── .env                         ← Environment variables (never commit)
├── package.json                 ← Project config + npm scripts
└── src/
    ├── app.js                   ← Express app config (middleware + routes)
    ├── config/
    │   └── db.js                ← MongoDB connection singleton
    ├── constants/
    │   ├── responseConstants.js ← HTTP codes, messages, roles
    │   └── dbCollections.js     ← MongoDB collection name strings
    ├── features/
    │   └── users/
    │       ├── user.route.js    ← URL definitions
    │       ├── user.controller.js ← HTTP ↔ function translation
    │       ├── user.service.js  ← Business logic & validation
    │       └── user.repository.js ← DB queries only
    └── shared/
        ├── errors/
        │   └── AppError.js      ← Custom error class
        ├── middlewares/
        │   └── errorHandler.js  ← Global error middleware
        └── utils/
            └── handlers.js      ← sendSuccess / sendError helpers
```

1.2 Architecture Diagram

Request flows top-down through each layer. Each layer talks only to the one directly below it.






```
Shared (used across all layers):
AppError.js   errorHandler.js   handlers.js
```

 Golden Rule Each layer has ONE job. No layer skips another. Data flows down, responses flow up.

2. package.json — Project Config

```
{
  "type": "module",
  "scripts": {
    "start": "node --env-file=.env server.js",
    "dev":   "nodemon --exec \"node --env-file=.env\" server.js"
  },
  "dependencies": {
    "express": "^5.2.1",
    "mongodb": "^7.2.0",
    "cors":    "^2.8.6"
  },
  "devDependencies": {
    "nodemon": "^3.1.14"
  }
}
```

Line-by-line explanation

Key	What it means
"type": "module"	Enables ES Module syntax (import/export) in all .js files. Without this, Node.js defaults to CommonJS (require/module.exports).
"start" script	Runs the server in production. --env-file=.env loads .env variables into process.env natively — no dotenv package needed (Node 20.6+).
"dev" script	Uses nodemon to restart server on file changes. nodemon --exec "node --env-file=.env" sets the runner to node with env file support.
express ^5.2.1	Express 5 is a major update. Key change: async errors in route handlers are caught automatically — no need to wrap every async function in try/catch just to call next(err).
mongodb ^7.2.0	MongoDB native driver (not Mongoose). Direct driver gives more control, less magic. We manage connection ourselves.
cors ^2.8.6	Cross-Origin Resource Sharing middleware. Allows browsers from different origins (e.g., your React app on port 3000) to call this API on port 5000.
nodemon (devDep)	Development-only tool. Not needed in production. devDependencies are excluded when deploying with npm install --production.

⚠ **Note:** Interview Q: Why use --env-file instead of dotenv? Because Node 20.6+ supports it natively. No extra dependency, same result.

3. server.js — The Entry Point

server.js has exactly one job: boot the application in the correct order. It does NOT configure anything.

```
import app from "./src/app.js";
import { connectDB } from "./src/config/db.js";

const PORT = process.env.PORT || 5000;   // fallback if .env missing

const startServer = async () => {
  try {
    await connectDB();   // ← DB MUST connect before HTTP starts

    app.listen(PORT, () => {
      console.log(`Server running on port ${PORT}`);
    });
  } catch (error) {
    console.error(error);
    process.exit(1);     // ← exit code 1 = crash (Docker/PM2 will restart)
  }
};

startServer();
```

Why this order matters

- await connectDB() first — if a request comes in before DB is ready, repository functions throw "Database connection not initialized"
- app.listen() second — only open the HTTP port after DB is confirmed
- process.exit(1) on failure — exit code 1 signals an error. Process managers (PM2, Docker, Kubernetes) restart the process automatically on non-zero exit

Why app is imported, not configured here

server.js imports the fully configured app from app.js. This separation means you can import app in unit tests without starting the actual server or connecting to DB.

⚠ **Note:** Interview Q: Why separate server.js and app.js? So tests can import app without binding a port or needing a real DB connection.

4. src/app.js — Express App Configuration

```
import express from "express";
import cors from "cors";
import userRoutes from "./features/users/user.route.js";
import { errorHandler } from "./shared/middlewares/errorHandler.js";
import { HTTP_STATUS } from "./constants/responseConstants.js";
import { AppError } from "./shared/errors/AppError.js";  // ← must import!

const app = express();

// ── Middleware ──────────────────────────────────
app.use(express.json());   // parse JSON request bodies
app.use(cors());           // allow cross-origin requests

// ── Health check ────────────────────────────────
app.get("/", (req, res) => {
  res.send("Welcome API Home");
});

// ── Feature routes ──────────────────────────────
app.use("/users", userRoutes);

// ── 404 handler (must be AFTER all routes) ──────
app.use((req, res, next) => {
  next(new AppError(`Cannot ${req.method} ${req.path}`, HTTP_STATUS.NOT_FOUND));
});

// ── Global error handler (must be LAST) ─────────
app.use(errorHandler);

export default app;
```

Middleware registration order — critical

Middleware	Why this position
express.json()	Must be before routes. Parses JSON body into req.body. Without this, req.body is undefined.
cors()	Before routes. Sets CORS headers on every response. If placed after routes, responses go out without CORS headers.
Feature routes	After middleware. app.use("/users", userRoutes) mounts all user routes under the /users prefix.
404 handler	After all routes. If no route matched, this catches it and throws a clean AppError with 404.
errorHandler	Absolutely last. Express identifies error middleware by its 4-parameter signature (err, req, res, next).

What express.json() does

Without it: req.body = undefined for POST/PUT requests.

With it: Express reads the raw JSON string from the request body, parses it, and attaches it as req.body.

What cors() does

When a browser on localhost:3000 calls an API on localhost:5000, the browser blocks the request (same-origin policy). cors() adds the Access-Control-Allow-Origin header to every response, telling the browser the cross-origin call is permitted.

⚠ **Note:** Bug in current code: AppError is used in the 404 handler but was not imported. Always import before using.

How app.use("/users", userRoutes) works

This mounts the router at the /users prefix. Inside userRoutes, the routes are defined as / and /:id — Express combines them: /users + / = /users, /users + /:id = /users/:id.

5. src/config/db.js — MongoDB Connection Singleton

```
import { MongoClient } from "mongodb";

let client;  // module-level variable — persists across all imports

export async function connectDB() {
  if (client) return client;    // already connected — reuse

  client = new MongoClient(process.env.MONGODB_URI);
  await client.connect();
  return client;
}

export function getDB(dbName = process.env.DB_NAME) {
  if (!client) {
    throw new Error("Database connection not initialized");
  }
  if (!dbName) throw new Error("Invalid DB");
  return client.db(dbName);
}
```

The Singleton Pattern

let client is declared at module level. In Node.js, modules are cached after the first import — every file that imports db.js gets the same module instance, and therefore the same client variable.

Function	Purpose & behaviour
connectDB()	Called ONCE at startup in server.js. If called again (e.g., by accident), the if (client) guard returns the existing connection immediately — no duplicate connections.
getDB()	Called in every repository function. Returns client.db(dbName) — a lightweight reference to the specific database, not a new connection. Throws if connectDB() was never called.

Why not create a new MongoClient per request?

MongoDB connections are expensive to create (TCP handshake, auth, etc.). Creating one per request would slow every API call by ~50–100ms and quickly exhaust MongoDB's connection limit. The singleton keeps one connection pool alive and reuses it.

⚠ **Note:** Interview Q: What is the singleton pattern? An object that is created once and shared everywhere. The module-level let client is the singleton instance.

6. src/constants/ — Shared Constants

6.1 responseConstants.js

```
// HTTP Status Codes — no more magic numbers
export const HTTP_STATUS = {
  OK:                   200,
  CREATED:              201,
  NO_CONTENT:           204,
  BAD_REQUEST:          400,
  UNAUTHORIZED:         401,
  FORBIDDEN:            403,
  NOT_FOUND:            404,
  CONFLICT:             409,
  UNPROCESSABLE_ENTITY: 422,
  INTERNAL_SERVER_ERROR: 500,
};

// Reusable response messages
export const MESSAGES = {
  USER_CREATED:      "User created successfully",
  USER_NOT_FOUND:    "User not found",
  ALREADY_EXISTS:    "User already exists",
  USER_LIST_FETCHED: "User list fetched successfully",
};

// Role constants (for future auth/RBAC)
export const USER_ROLES = {
  ADMIN: "admin",
  USER:  "user",
};
```

✓ **Correct:** HTTP_STATUS.CONFLICT is self-documenting. 409 is a magic number you have to remember.

✗ **Wrong:** throw new AppError("Already exists", 409) — What is 409? You have to look it up.

✓ **Correct:** throw new AppError(MESSAGES.ALREADY_EXISTS, HTTP_STATUS.CONFLICT) — Clear at a glance.

6.2 dbCollections.js

```
export const DB_COLLECTIONS = {
  USERS: "users",
};

// Usage in repository:
// db.collection(DB_COLLECTIONS.USERS).find({})
```

The string "users" was repeated in 3 places in user.repository.js. If you rename the MongoDB collection, you change it here — one place — instead of hunting through files.

⚠ **Note:** Bug found: user.repository.js imports from "../../constants/dbCOllections.js" (capital O). File is named dbCollections.js (lowercase o). This causes a crash on case-sensitive filesystems (Linux, production). Fix: use exact case.

7. user.route.js — URL Map

```
import express from "express";
import * as userController from "./user.controller.js";

const router = express.Router();

router.get("/",    userController.getUsers);      // GET  /users
router.post("/",   userController.createUser);    // POST /users

// Planned (currently commented out):
// router.get("/:id",    userController.getUserById);
// router.put("/:id",    userController.updateUser);
// router.delete("/:id", userController.deleteUser);

export default router;
```

What is express.Router()?

Router() creates a mini Express app — a self-contained group of routes. It has no idea where it will be mounted. In app.js, app.use("/users", userRoutes) gives it its prefix. This means: inside the router, write / — not /users/.

import * as userController

This imports every named export from user.controller.js into a single object. So userController.getUsers, userController.createUser, etc. Clean and explicit — you can see all handlers at a glance.

Why route files have no logic

The route file's only job is the mapping: which HTTP verb + URL pattern triggers which function. If you put if/else or validation here, it becomes hard to test and hard to reuse.

HTTP Method	What it means / when to use it
GET	Retrieve data. No body. Safe and idempotent (calling it 10 times has same result as calling it once).
POST	Create new resource. Has request body. Not idempotent (calling it 10 times creates 10 records).
PUT	Replace entire resource. Has body. Idempotent.
PATCH	Partial update of resource. Has body. Common alternative to PUT for updates.
DELETE	Remove resource. No body. Idempotent.

8. user.controller.js — HTTP Translator

```
import { HTTP_STATUS, MESSAGES } from "../../constants/responseConstants.js";
import { sendSuccess } from "../../shared/utils/handlers.js";
import * as userService from "./user.service.js";

export const getUsers = async (req, res, next) => {
  try {
    const users = await userService.getUsers();
    sendSuccess({
      res,
      statusCode: HTTP_STATUS.OK,
      data: users,
      message: MESSAGES.USER_LIST_FETCHED,
    });
  } catch (error) {
    next(error);   // ← pass to global errorHandler
  }
};

export const createUser = async (req, res, next) => {
  try {
    const user = await userService.createUser(req.body);
    sendSuccess({
      res,
      statusCode: HTTP_STATUS.CREATED,
      data: user,
      message: MESSAGES.USER_CREATED,
    });
  } catch (error) {
    next(error);
  }
};
```

Controller's one job

Read from req → call service → send response. It knows about HTTP (req, res, status codes) but knows nothing about MongoDB or business rules.

Why next(error) instead of res.status(500).json(...)

Calling next(error) passes the error to Express's error-handling pipeline. The global errorHandler middleware handles it uniformly. If every controller did its own error response, you'd have inconsistent error formats across endpoints.

req, res, next — what are they?

Parameter	What it is

req (Request)	The incoming HTTP request. Contains: req.body (parsed JSON), req.params (URL params like :id), req.query (query strings like ?page=1), req.headers.
res (Response)	The outgoing HTTP response. Used to send data back: res.json(), res.status(), res.send(). Once you call one of these, the response is sent — you cannot send again.
next	A function that passes control to the next middleware. next() = continue normally. next(error) = jump to error handler. Always call one of: res.json/send OR next(error).

Express 5 note on async

In Express 4, an unhandled promise rejection in an async route handler would crash the server silently. In Express 5, async errors are caught automatically — the try/catch + next(error) pattern is still good practice for explicitness, but the safety net exists.

⚠ **Note:** Interview Q: What happens if you forget next(error) in a controller? In Express 4 — the request hangs forever (no response sent). In Express 5 — async errors are caught, but sync errors still need next(error).

9. user.service.js — Business Logic Layer

```
import { HTTP_STATUS, MESSAGES } from "../../constants/responseConstants.js";
import { AppError } from "../../shared/errors/AppError.js";
import * as userRepository from "./user.repository.js";

export const getUsers = async () => {
  return userRepository.findAll();
};

export const createUser = async (userData) => {
  // Rule 1: name is required
  if (!userData.name) {
    throw new AppError("Name is required", HTTP_STATUS.BAD_REQUEST);
  }

  // Rule 2: no duplicate emails
  const existingUser = await userRepository.findByEmail(userData.email);
  if (existingUser) {
    throw new AppError(MESSAGES.ALREADY_EXISTS, HTTP_STATUS.CONFLICT);
  }

  return userRepository.create(userData);
};
```

Why business logic belongs here, not in the controller

Services are pure JavaScript — they receive plain data and return plain data. They have no knowledge of req or res. This means:

- Services can be called from cron jobs, queue workers, CLI scripts — not just HTTP routes
- Services can be unit-tested without spinning up Express
- A PM can describe a rule in English ("users must be unique by email") — you translate that directly into service code

Why throw AppError, not res.status()

The service does not have access to res — and it shouldn't. Throwing AppError lets the error bubble up through the controller's catch block, then to the global errorHandler, which sends the response. Clean separation.

⚠ **Note:** Interview Q: Where should validation live — controller or service? Service. Controllers are HTTP glue. Services own the rules. If you validate in the controller, the rules are tied to HTTP and can't be reused.

10. user.repository.js — Data Access Layer

```
import { getDB } from "../../config/db.js";
import { DB_COLLECTIONS } from "../../constants/dbCollections.js";

export const findAll = async () => {
  const db = getDB();
  return db.collection(DB_COLLECTIONS.USERS).find({}).toArray();
};

export const findByEmail = async (email) => {
  const db = getDB();
  return db.collection(DB_COLLECTIONS.USERS).findOne({ email });
};

export const create = async (userData) => {
  const db = getDB();
  const result = await db
    .collection(DB_COLLECTIONS.USERS)
    .insertOne(userData);
  return { _id: result.insertedId, ...userData };
};
```

Repository Pattern — why it exists

The repository is the only file that speaks MongoDB. If you switch from MongoDB to PostgreSQL tomorrow, you rewrite only repository files. Everything above — service, controller, route — stays unchanged.

Common MongoDB driver operations

Operation	Code
Find all	collection.find({}).toArray()
Find with filter	collection.find({ age: { $gt: 18 } }).toArray()
Find one	collection.findOne({ email })
Insert one	const r = await collection.insertOne(doc) → r.insertedId
Update one	collection.updateOne({ _id }, { $set: { name } })
Delete one	collection.deleteOne({ _id })
Count	collection.countDocuments({ status: "active" })

What .find({}) returns

.find({}) returns a cursor — a pointer to the results, not the results themselves. You must call .toArray() to fetch all documents into memory, or iterate with for await...of cursor for large datasets.

Why create() returns the inserted document

insertOne() returns an object with insertedId (the new MongoDB _id). We spread userData back with the new _id so the caller gets the full document without a second DB round-trip.

11. shared/errors/AppError.js — Custom Error Class

```
export class AppError extends Error {
  constructor(message, statusCode) {
    super(message);        // sets this.message
    this.statusCode = statusCode;
  }
}

// Usage in service:
throw new AppError("User not found", HTTP_STATUS.NOT_FOUND);

// Caught by errorHandler:
// err.message    → "User not found"
// err.statusCode → 404
```

Why extend Error?

By extending the native Error class, AppError gets the full stack trace, instanceof checks work (err instanceof AppError), and it integrates naturally with try/catch and Express error handling.

The flow

1. Service throws: throw new AppError("User not found", 404)
2. Controller catches it in try/catch and calls next(error)
3. Express routes the error to errorHandler (4-param middleware)
4. errorHandler reads err.statusCode and err.message and sends JSON response

⚠ **Note:** Interview Q: What is the difference between throw new Error() and throw new AppError()? Error has no statusCode. AppError carries an HTTP status code so errorHandler can set the correct HTTP response status.

12. shared/middlewares/errorHandler.js — Global Error Middleware

```
import { HTTP_STATUS } from "../../constants/responseConstants.js";
import { sendError } from "../utils/handlers.js";

export const errorHandler = (err, req, res, next) => {
  const message    = err.message || "Internal Server Error";
  const statusCode = err.statusCode || HTTP_STATUS.INTERNAL_SERVER_ERROR;

  console.error(err);   // log full error + stack trace

  sendError({ res, message, statusCode });
};
```

The 4-parameter signature

Express identifies error-handling middleware by the number of parameters. Any middleware with exactly 4 parameters (err, req, res, next) is an error handler. Declare it last in app.js.

Fallback defaults

Expression	What it handles
err.message || "Internal Server Error"	If the error has no message (e.g., a plain throw), defaults to a safe generic message.
err.statusCode || 500	AppError instances have statusCode. Native Error objects do not. Falls back to 500 for unexpected errors.

Why log with console.error(err)?

This logs the full error object including the stack trace. In production you'd send this to a logging service (Datadog, Sentry, CloudWatch). Never expose raw stack traces to the client — only safe messages.

⚠ **Note:** Interview Q: What makes Express treat a middleware as an error handler? The function signature must have exactly 4 parameters. Express does not care about the name — only the parameter count.

13. shared/utils/handlers.js — Response Helpers

```
export const sendSuccess = ({ res, data, message = "Success", statusCode = 200 })
=> {
  res.status(statusCode).json({
    success: true,
    message,
    data,
  });
};

export const sendError = ({ res, message = "Internal Server Error", statusCode =
500 }) => {
  res.status(statusCode).json({
    success: false,
    message,
  });
};
```

Why centralise response formatting?

Without these helpers, every controller would write res.status(200).json({ success: true, ... }). If you later want to add a timestamp or requestId to every response, you change one function — not every controller.

The response shape

```
// Success response (GET /users):
{
  "success": true,
  "message": "User list fetched successfully",
  "data": [ ... ]
}

// Error response (POST /users with duplicate email):
{
  "success": false,
  "message": "User already exists"
}
```

res.status().json() — how it works

res.status(statusCode) sets the HTTP status code. .json(obj) serialises the object to JSON, sets Content-Type: application/json, and sends the response. These can be chained because .status() returns the res object.

14. Middleware — Core Express Concept

Middleware is any function with the signature (req, res, next). Express processes middleware in registration order. Each middleware can: modify req/res, end the request, or call next() to pass to the next middleware.

Middleware types

Type	Signature & example
Application middleware	app.use(fn) — runs on every request. Example: express.json(), cors()
Route middleware	app.use("/users", fn) — runs only for /users routes
Error middleware	(err, req, res, next) — 4 params, runs only when next(error) is called
Router middleware	router.use(fn) — scoped to a specific router
Built-in	express.json(), express.urlencoded(), express.static()
Third-party	cors(), morgan, helmet, rate-limiter

Middleware you should add in a real project

helmet — Security headers

```
import helmet from "helmet";
app.use(helmet());
// Sets: X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security, etc.
// Install: npm install helmet
```

morgan — Request logging

```
import morgan from "morgan";
app.use(morgan("dev"));
// Logs: GET /users 200 12ms
// Install: npm install morgan
```

express-rate-limit — Protect against abuse

```
import rateLimit from "express-rate-limit";
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000,  // 15 minutes
  max: 100,                   // max 100 requests per window
});
app.use(limiter);
// Install: npm install express-rate-limit
```

requestLogger — Custom middleware example

```
// src/shared/middlewares/requestLogger.js
```

```
export const requestLogger = (req, res, next) => {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.path}`);
  next();   // MUST call next() or request hangs
};

// In app.js:
app.use(requestLogger);
```

⚠ **Note:** Interview Q: What happens if you forget to call next() in middleware? The request hangs — no response is ever sent. The client waits until timeout.

15. ES Modules vs CommonJS

Feature	ES Modules (this project) vs CommonJS
Import syntax	import x from "y" vs const x = require("y")
Export syntax	export const fn = ... vs module.exports = { fn }
File extension	Keep .js with "type":"module" in package.json
Async at top level	Allowed in ESM vs Not allowed in CJS
When to use	New projects — ESM is the standard. Legacy code still uses CJS.

Why .js extension is required in imports

```
// ✓ Correct — explicit .js extension required in ESM
import { connectDB } from "./src/config/db.js";

// ✗ Wrong — works in CommonJS but fails in ESM
import { connectDB } from "./src/config/db";
```

Node.js ESM does not auto-resolve extensions. Always include .js in relative imports.

16. Environment Variables — .env & process.env

```
# .env
PORT=5000
MONGODB_URI=mongodb://localhost:27017/
DB_NAME='demoDB'
```

How variables get into process.env

The start script uses node --env-file=.env server.js. Node reads .env and populates process.env before the app starts. No dotenv package needed in Node 20.6+.

Accessing variables

```
process.env.PORT          // "5000" (always a string)
process.env.MONGODB_URI   // "mongodb://localhost:27017/"
process.env.DB_NAME       // "demoDB"

// Fallback pattern:
const PORT = process.env.PORT || 5000;
```

Rule	Reason
Never commit .env to git	Contains secrets. .gitignore already excludes it.
Use .env.example in repo	Shows shape of config without real values. Teammates copy it.
Validate at startup	If MONGODB_URI is missing, crash early with a clear message rather than mysterious failures later.

Validation pattern (recommended addition)

```
// src/config/validateEnv.js
export function validateEnv() {
  const required = ["MONGODB_URI", "DB_NAME", "PORT"];
  for (const key of required) {
    if (!process.env[key]) {
      throw new Error(`Missing required env variable: ${key}`);
    }
  }
}

// In server.js — call before anything else:
validateEnv();
```

17. Full Request Lifecycle — POST /users

Trace exactly what happens when a POST /users request hits the server:

```
POST /users
Body: { "name": "Sivaraj", "email": "siva@example.com" }

Step 1 → express.json() middleware
         Reads raw body → parses JSON → req.body = { name, email }

Step 2 → cors() middleware
         Adds CORS headers to response

Step 3 → app.use("/users", userRoutes)
         Path matches → hands off to userRoutes router

Step 4 → user.route.js: router.post("/", userController.createUser)
         Method + path match → calls createUser controller

Step 5 → user.controller.js: createUser(req, res, next)
         Reads req.body → calls userService.createUser(req.body)

Step 6 → user.service.js: createUser(userData)
         Validates: name present? email unique?
         → calls userRepository.create(userData)

Step 7 → user.repository.js: create(userData)
         getDB() → collection.insertOne(userData)
         → returns { _id: ..., name, email }

Step 8 → back in service → returns result to controller

Step 9 → back in controller → sendSuccess({ res, 201, data, message })
         → res.status(201).json({ success: true, ... })

Response:
{
  "success": true,
  "message": "User created successfully",
  "data": { "_id": "...", "name": "Sivaraj", "email": "siva@example.com" }
}
```

Error path — duplicate email

```
Step 6 → service finds existingUser → truthy
         throws new AppError("User already exists", 409)

Step 5 → controller catch block → next(error)
```

```
Express → skips all remaining routes
        → finds 4-param errorHandler middleware

errorHandler → err.statusCode = 409, err.message = "User already exists"
             → sendError({ res, 409, "User already exists" })

Response:
{ "success": false, "message": "User already exists" }
```

18. Interview Questions & Answers

Q: What is Express.js?

A minimal web framework for Node.js. It wraps Node's http module and adds routing, middleware, and request/response utilities. It does not enforce structure — you can organise code however you want.

Q: What is middleware in Express?

A function with (req, res, next) signature. Runs in order of registration. Can modify req/res, end the request cycle, or call next() to pass to the next middleware.

Q: What is the difference between app.use() and app.get()?

app.use() matches any HTTP method for the given path. app.get() matches only GET. app.use("/users") runs for GET, POST, PUT, DELETE to /users.

Q: Why is the error handler registered last?

Express processes middleware in registration order. If errorHandler is registered before routes, errors from routes never reach it.

Q: What is the 4-parameter signature for?

Express identifies error-handling middleware by exactly 4 parameters (err, req, res, next). Normal middleware has 3 (req, res, next). The count is the signal.

Q: What does next(error) do?

Passes the error to the nearest error-handling middleware. Express skips all remaining normal middleware and routes.

Q: Why separate controller, service, repository?

Single Responsibility Principle. Controller knows HTTP. Service knows business rules. Repository knows the database. Each can be tested independently. Each can change without breaking the others.

Q: What is the Repository pattern?

An abstraction over data access. All DB queries live in one place. If you change the database, only repositories change. Layers above are unaffected.

Q: What is the Singleton pattern? Where is it used here?

An object created once and reused. The MongoDB client in db.js is a singleton — one connection for the whole app lifetime, shared via module caching.

Q: What is CORS and why is it needed?

Cross-Origin Resource Sharing. Browsers block requests to different origins by default. cors() middleware adds the Access-Control-Allow-Origin header to allow the browser to proceed.

Q: How does Express 5 handle async errors differently from Express 4?

Express 5 catches rejected promises from async route handlers automatically. Express 4 required manual try/catch and next(err) call — a rejected promise in Express 4 would crash the server.

Q: What are ES Modules vs CommonJS?

ESM uses import/export syntax, requires .js extensions, allows top-level await. CJS uses require/module.exports. ESM is the modern standard. Enabled by "type":"module" in package.json.

Q: What does process.exit(1) mean?

Exit the Node process with code 1 (error). Code 0 = clean exit. Process managers (PM2, Docker, Kubernetes) restart the process if it exits with a non-zero code.

Q: Why put AppError in shared/errors/?

AppError is used across multiple layers — services throw it, errorHandler catches it. Shared code that crosses feature boundaries belongs in shared/.

Q: What is the difference between findOne and find?

findOne returns a single document or null. find returns a cursor (lazy iterator). You must call .toArray() on a cursor to get all results into memory.

Q: What does req.body contain?

The parsed request body. Only available after express.json() (for JSON) or express.urlencoded() (for form data) middleware runs. Without the middleware, req.body is undefined.

19. Quick Reference — Layer Responsibilities

File	One job only
server.js	Boot sequence: connectDB → listen → crash on failure
app.js	Configure Express: middleware → routes → 404 → errorHandler
user.route.js	URL map only: HTTP verb + path → controller function
user.controller.js	HTTP translation: read req → call service → send res
user.service.js	Business logic: validate → rules → call repository
user.repository.js	DB queries only: find/insert/update/delete
config/db.js	MongoDB singleton: one connection, shared everywhere
AppError.js	Custom error with statusCode — thrown by service, caught by errorHandler
errorHandler.js	Last middleware: catch all errors, send clean JSON response
handlers.js	sendSuccess / sendError — consistent response shape
responseConstants.js	HTTP_STATUS, MESSAGES, USER_ROLES — no magic numbers
dbCollections.js	DB_COLLECTIONS — single source of truth for collection names

Rules to never break

- Controller never calls getDB() directly
- Service never touches req or res
- Repository never throws AppError
- errorHandler always registered last in app.js
- connectDB() always before app.listen()
- Constants over magic numbers and hardcoded strings

Build order for a new feature

5. repository — closest to DB, build first
6. service — add business rules on top
7. controller — wrap in HTTP
8. route — define the URL
9. app.js — mount the route